

Java SE 17 Developer Certification

Complete Revision Guide

Contents

1	Java Fundamentals	3
1.1	Enums	3
1.2	Switch Statements	4
2	Strings & StringBuilder	5
2.1	String Immutability	5
2.2	StringBuilder	5
2.3	String Pool & intern()	6
3	Date & Time API (java.time)	7
3.1	Duration and Period Formatting	7
3.2	Daylight Saving Time (DST)	7
3.3	truncatedTo() Method	7
4	Streams & Optional	9
4.1	reduce() Methods	9
4.2	ExecutorService Methods	10
4.3	dropWhile() Operation	10
5	Serialization	11
5.1	Serialization Rules	11
5.1.1	Field Behavior	11
5.1.2	Custom Serialization	11
5.2	Common Serialization Issues	11
6	Concurrency	13
6.1	Thread States	13
6.2	InterruptedException	13
6.3	volatile vs AtomicInteger	14
6.4	Lock Methods	14
7	Java Modules (JPMS)	15
7.1	Module Types	15
7.2	Automatic Module Naming	15
7.3	Migration Strategies	15
8	Interfaces & Sealed Classes	17
8.1	Interface Methods	17
8.2	Sealed Interfaces	17

9 Quick Reference Guide	19
10 Practice Tips	21
11 Key Takeaways	23

1 Java Fundamentals

1.1 Enums

Key Points:

- Enums are special classes that represent fixed set of constants
- Can have fields, constructors (private), and methods
- Constructors are **private** by default
- Use `values()` to get all enum constants
- Use `valueOf(String)` to get enum constant by name
- `ordinal()` returns position of enum constant (0-based)

```
1 public enum Desig {  
2     CEO('A'), CMO('B'), CTO('C'), CFO('D');  
3     char c;  
4     private Desig(char c) { this.c = c; }  
5 }  
6 // Usage: Desig.valueOf("CMO") returns Desig.CMO
```

Listing 1: Enum Example

Important

- Enum constants are **public static final** by default
- Enum constructors are **always private**
- Enum can implement interfaces but cannot extend classes
- Use `EnumSet` and `EnumMap` for better performance

1.2 Switch Statements

Enhanced Switch (Java 14+):

- Uses arrow -> (no fall-through)
- Multiple case labels with commas
- No `break` needed
- Can be used as expression (returns value)

```
1 switch (rank) {  
2     case 1,4 -> System.out.println("Range1"); //      No fall-through  
3     case 5,8 -> System.out.println("Range2");  
4     default -> System.out.println("Default");  
5 }  
6  
7 // Switch expression  
8 int result = switch (day) {  
9     case "MON", "FRI" -> 6;  
10    case "WED" -> 4;  
11    default -> 0;  
12 };
```

Listing 2: Enhanced Switch

Traditional Switch:

- Uses colon :
- Fall-through unless `break` is used
- Duplicate case labels cause compilation error

```
1 switch (y) {  
2     case 2:  
3         y++; //      Falls through to next case if no break  
4     case 1:  
5         y++;  
6         break; // Need break to prevent fall-through  
7 }
```

Listing 3: Traditional Switch

Tip

Always use `default` case in switch statements to handle unexpected values.

2 Strings & StringBuilder

2.1 String Immutability

Important

Strings are **immutable** - once created, cannot be changed.

- Methods like `replace()`, `concat()`, `substring()` return **new** Strings
- Original String remains unchanged
- String methods do NOT modify the original object

```
1 String a = "Hello! Java";
2 a.replace("Hello!", "Welcome!"); // a is NOT changed
3 System.out.println(a.indexOf("Java")); // Still returns 7
4
5 String b = a.replace("Hello!", "Welcome!"); // New String created
6 // b = "Welcome! Java"
7 // a = "Hello! Java" (unchanged)
```

Listing 4: String Immutability

2.2 StringBuilder

Tip

Use **StringBuilder** when you need to modify strings frequently.

- **Mutable** - modifications change the object
- Has `indexOf()` method
- Methods like `append()`, `insert()`, `delete()` modify the object
- Not thread-safe (use `StringBuffer` for thread-safety)

```
1 StringBuilder b = new StringBuilder("Hello! Java");
2 b.indexOf("Java"); // Returns 7
3 b.append(" World"); // Modifies the same object
4 System.out.println(b); // "Hello! Java World"
```

Listing 5: StringBuilder Example

2.3 String Pool & intern()

Key Concepts:

- String literals are automatically interned (stored in string pool)
- `intern()` returns canonical representation from string pool
- `==` compares references (memory addresses)
- `equals()` compares content (value)

```
1 String s1 = "---";
2 String s2 = "---";
3 String s3 = new String("---");
4
5 // Reference comparisons
6 s1 == s2;    //      true (same pool reference)
7 s1 == s3;    //      false (different objects)
8 s1.intern() == s2.intern(); //      true (both pool references)
9 s1.intern() == s3.intern(); //      true (s3.intern() returns pool reference)
```

Listing 6: String Pool Example

Method	Description
<code>equals()</code>	Compares content (case-sensitive)
<code>equalsIgnoreCase()</code>	Compares content (case-insensitive)
<code>==</code>	Compares references
<code>intern()</code>	Returns pooled reference

Table 1: String Comparison Methods

3 Date & Time API (java.time)

3.1 Duration and Period Formatting

ISO-8601 Duration Format:

- Duration → PTnHnMnS (Period of Time)
- Period → PnYnMnD (Period)
- LocalDate → No time component
- LocalTime → No date component
- ZonedDateTime → Date + Time + TimeZone

```
1 // Durations (time-based)
2 Duration.ofMillis(5000) // PT5S
3 Duration.ofSeconds(60) // PT1M
4 Duration.ofMinutes(30) // PT30M
5 Duration.ofHours(2) // PT2H
6 Duration.ofDays(1) // PT24H
7
8 // Periods (date-based)
9 Period.ofDays(6) // P6D
10 Period.of(1, 2, 3) // P1Y2M3D (1 year, 2 months, 3 days)
11 Period.ofMonths(3) // P3M
```

Listing 7: Duration and Period Examples

3.2 Daylight Saving Time (DST)

Important

During DST transitions:

- **Fall-back:** Clocks go backward (hour repeats)
- **Spring-forward:** Clocks go forward (hour skipped)
- **ZonedDateTime** handles these transitions automatically

```
1 ZoneId zoneID = ZoneId.of("America/Chicago");
2 ZonedDateTime zdt = ZonedDateTime.of(
3     LocalDate.of(2021, 11, 7),
4     LocalTime.of(1, 30),
5     zoneID
6 );
7
8 ZonedDateTime anHourLater = zdt.plusHours(1);
9 // 1:30 AM CDT + 1 hour = 1:30 AM CST (due to DST fall-back)
10 // Hour remains 1, but offset changes from UTC-5 to UTC-6
```

Listing 8: DST Handling

3.3 truncatedTo() Method

```

1 Instant now = Instant.now();           // 2021-01-12T21:58:18.817Z
2 now.truncatedTo(ChronoUnit.MINUTES);   // 2021-01-12T21:58:00Z
3 // Removes seconds and nanoseconds, rounds DOWN
4
5 ZonedDateTime zdt = ZonedDateTime.now();
6 zdt.truncatedTo(ChronoUnit.HOURS);      // Rounds to start of hour
7 zdt.truncatedTo(ChronoUnit.DAYS);       // Rounds to start of day

```

Listing 9: truncatedTo Example

ChronoUnit	Effect
NANOS	Removes milliseconds, microseconds
MILLIS	Removes microseconds, nanoseconds
SECONDS	Removes milliseconds, nanoseconds
MINUTES	Removes seconds, nanoseconds
HOURS	Removes minutes, seconds, nanoseconds
DAYS	Removes time component entirely

Table 2: ChronoUnit truncation levels

4 Streams & Optional

4.1 reduce() Methods

Two Forms of reduce():

- **With identity** → Returns T (not Optional)
- **Without identity** → Returns Optional<T>

```
1 // With identity - returns actual value
2 int sum = stream.reduce(0, Integer::sum); // Returns int
3
4 // Without identity - returns Optional
5 Optional<Integer> sum = stream.reduce(Integer::sum); // Returns Optional<
6     Integer>
7
8 // Handling Optional
9 int sum = stream.reduce(Integer::sum).orElse(0); // Use orElse()
10 int sum = stream.reduce(Integer::sum).get(); // Use get() (unsafe!)
```

Listing 10: reduce() Examples

Important

The version **without identity** returns Optional because:

- If stream has elements → returns Optional.of(result)
- If stream is **empty** → returns Optional.empty()
- No default value to fall back on

4.2 ExecutorService Methods

Method	Executes All?	Returns
invokeAll(collection)	Yes	List<Future<T>
invokeAny(collection)	Yes	T (first completed)
submit(callable)	No (single task)	Future<T>
execute(runnable)	No (single task)	void

Table 3: ExecutorService Methods

```
1 Set<Callable<String>> workers = new HashSet<>();
2
3 // invokeAll - executes ALL tasks
4 List<Future<String>> futures = executorService.invokeAll(workers);
5 for (Future<String> future : futures) {
6     System.out.println(future.get()); // Prints ALL results
7 }
8
9 // invokeAny - executes ALL but returns only first completed
10 String result = executorService.invokeAny(workers);
11 System.out.println(result); // Only ONE result
12
13 // submit - executes ONE task
14 Future<String> future = executorService.submit(workers.iterator().next());
15 String result = future.get();
```

Listing 11: ExecutorService Examples

4.3 dropWhile() Operation

```
1 // Drops elements while predicate is true
2 // Returns remaining elements
3 Arrays.stream(Desig.values()).dropWhile(s -> s.equals(Desig.CM0));
4 // Only works on ORDERED streams
5 // Does NOT execute unless terminal operation is called
```

Listing 12: dropWhile Example

5 Serialization

5.1 Serialization Rules

Serialization Summary:

- Class must implement `Serializable`
- All superclasses must be serializable
- Subclass must also implement `Serializable`
- `transient` fields are NOT serialized
- `static` fields are NOT serialized

5.1.1 Field Behavior

Modifier	Serialized?	Notes
<code>transient</code>	No	Skipped during serialization
<code>static</code>	No	Belongs to class, not object
<code>non-transient instance</code>	Yes	Default behavior
<code>final</code>	Yes	Can be serialized

5.1.2 Custom Serialization

```
1 public class Product implements Serializable {
2     private static float averagePrice = 2.99f;
3     private String description;
4     private transient float price;
5
6     // Custom deserialization
7     private void readObject(ObjectInputStream in)
8         throws IOException, ClassNotFoundException {
9         in.defaultReadObject(); // Read non-transient fields
10        price = averagePrice;   // Initialize transient field
11    }
12 }
```

Listing 13: Custom `readObject()`

5.2 Common Serialization Issues

Important

Serialization Exceptions:

- `NotSerializableException` - Class doesn't implement `Serializable`
- `ClassNotFoundException` - Class not found during deserialization
- `InvalidClassException` - `SerialVersionUID` mismatch

```
1 class Car implements Serializable { } //      Serializable
2
3 class LuxuryCar extends Car { }          //      NOT Serializable
4 // Throws NotSerializableException
5
6 // Fix:
7 class LuxuryCar extends Car implements Serializable { } //
```

Listing 14: Serialization Error Example

Tip

Always declare `serialVersionUID` to maintain version compatibility: `private static final long serialVersionUID = 1L;`

6 Concurrency

6.1 Thread States

Thread States & CPU Usage:

State	Description	Consumes CPU?
RUNNABLE	Ready to run / Running	Yes
WAITING	Waiting indefinitely	No
TIMED_WAITING	Waiting with timeout	No
BLOCKED	Waiting for lock	No
TERMINATED	Completed execution	No

6.2 InterruptedException

Important

InterruptedException is a **checked exception** - must be handled!

```
1 // Option 1: Try-catch
2 try {
3     Thread.sleep(1000);
4 } catch (InterruptedException e) {
5     Thread.currentThread().interrupt(); // Restore interrupted status
6     // Handle interruption
7 }
8
9 // Option 2: Declare in method signature
10 public void myMethod() throws InterruptedException {
11     Thread.sleep(1000);
12 }
```

Listing 15: Handling InterruptedException

6.3 volatile vs AtomicInteger

Feature	volatile	AtomicInteger
Visibility	Yes	Yes
Atomicity	No	Yes
Performance	Faster	Slightly slower
Use Case	Single read/write	Complex operations

```
1 volatile int x = 1;
2 x++; // NOT atomic (read-increment-write)
3 // Three separate operations that can be interleaved
4
5 AtomicInteger xObj = new AtomicInteger(1);
6 xObj.getAndIncrement(); // Atomic operation
7 xObj.incrementAndGet(); // Atomic operation
8 xObj.compareAndSet(1, 2); // Atomic operation
```

Listing 16: volatile vs AtomicInteger

6.4 Lock Methods

Method	Returns	Blocks/Waits?
lock()	void	Yes - waits indefinitely
tryLock()	boolean	No - returns immediately
tryLock(time, unit)	boolean	Yes - waits specified time
lockInterruptibly()	void	Yes - interruptible

Table 4: Lock Methods Comparison

```
1 Lock lock = new ReentrantLock();
2
3 // lock() - blocks until acquired
4 lock.lock();
5 try {
6     // Critical section
7 } finally {
8     lock.unlock();
9 }
10
11 // tryLock() - non-blocking
12 if (lock.tryLock()) {
13     try {
14         // Critical section
15     } finally {
16         lock.unlock();
17     }
18 } else {
19     // Could not acquire lock
20 }
21
22 // tryLock() with timeout
23 if (lock.tryLock(10, TimeUnit.SECONDS)) {
24     // Lock acquired within 10 seconds
25 }
```

Listing 17: Lock Examples

7 Java Modules (JPMS)

7.1 Module Types

Three Types of Modules:

1. **Named Modules** - Have `module-info.java`
2. **Automatic Modules** - JAR on module path without `module-info.java`
3. **Unnamed Modules** - Classes on classpath

Module Type	Location	Has module-info?
Named Module	Module Path	Yes
Automatic Module	Module Path	No (regular JAR)
Unnamed Module	Classpath	No

7.2 Automatic Module Naming

Important

How JAR becomes Automatic Module:

1. Place JAR on module path (no `module-info.java`)
2. Module name derived from JAR filename
3. Remove extension (`.jar`)
4. Remove version number
5. Replace invalid characters with dots

```
1 display-ascii-0.2.jar      display.ascii
2 my-library-1.0.0.jar      my.library
3 commons-lang3-3.12.0.jar   commons.lang3
4 guava-31.1-jre.jar        guava
5 log4j-core-2.17.1.jar     log4j.core
```

Listing 18: Automatic Module Naming Examples

7.3 Migration Strategies

Strategy	Start With	Result
Bottom-Up	Leaf modules first	All modules on module path
Top-Down	Application first	Mix of module types

Tip

Bottom-up migration is preferred because it:

- Moves **every module** to the module path
- Better encapsulation
- Cleaner dependency management

8 Interfaces & Sealed Classes

8.1 Interface Methods

Method Type	On Interface	On Instance	Override
Abstract	No	Yes	Yes
Default	No	Yes	Yes
Static	Yes (interface name)	No	No
Private	No	No	No

Table 5: Interface Method Access

```
1 interface IFace {
2     void m1(); // Abstract
3     default void m2() { } // Default (instance)
4     static void m3() { } // Static (belongs to interface)
5     private void m4() { } // Private (only inside interface)
6 }
7
8 // Correct Usage:
9 IFace.m3(); // Static method
10 new MyC().m2(); // Default method (on instance)
11 IFace obj = new MyC();
12 obj.m2(); // Default method (on instance)
13
14 // Wrong Usage:
15 IFace.m2(); // Cannot call default on interface
16 obj.m3(); // Cannot call static on instance
17 IFace.m4(); // Private - not accessible
```

Listing 19: Interface Methods

8.2 Sealed Interfaces

Sealed Classes/Interfaces Rules:

- Must declare all permitted subtypes in `permits` clause
- Permitted subtypes must declare one of:
 - `sealed` - continues hierarchy
 - `non-sealed` - opens hierarchy
 - `final` - ends hierarchy

```
1 abstract sealed interface SInt permits Story, Art {
2     default String getTitle() { return "Book Title"; }
3 }
4
5 // Correct sub-interfaces
6 non-sealed interface Story extends SInt { }
7 non-sealed interface Art extends SInt { }
8
9 // Wrong - missing modifier
10 interface Story extends SInt { } // Compilation error
11
12 // Wrong - class when interface expected
```

```
13 class Story implements SInt { } // Compilation error
```

Listing 20: Sealed Interface Example

9 Quick Reference Guide

String Methods Summary

Method	Description
<code>charAt(int)</code>	Returns character at index
<code>concat(String)</code>	Concatenates strings
<code>contains(CharSequence)</code>	Checks if substring exists
<code>endsWith(String)</code>	Checks suffix
<code>equals(Object)</code>	Compares content
<code>indexOf(String)</code>	Returns first occurrence index
<code>intern()</code>	Returns pooled reference
<code>isEmpty()</code>	Checks if length is 0
<code>length()</code>	Returns length
<code>replace(old, new)</code>	Replaces characters
<code>substring(int)</code>	Returns substring
<code>toLowerCase()</code>	Converts to lowercase
<code>toUpperCase()</code>	Converts to uppercase
<code>trim()</code>	Removes whitespace

Table 6: String Methods

Builder/StringBuffer Methods

Method	Description
<code>append(type)</code>	Appends value
<code>insert(index, type)</code>	Inserts at position
<code>delete(start, end)</code>	Deletes characters
<code>replace(start, end, str)</code>	Replaces substring
<code>reverse()</code>	Reverses sequence
<code>indexOf(String)</code>	Returns occurrence index
<code>substring(start)</code>	Returns substring
<code>toString()</code>	Converts to String

Table 7: StringBuilder Methods

DateTime Formatting

Pattern	Description
<code>yyyy</code>	Year (4 digits)
<code>MM</code>	Month (2 digits)
<code>dd</code>	Day (2 digits)
<code>HH</code>	Hour (24-hour)
<code>mm</code>	Minute
<code>ss</code>	Second
<code>SSS</code>	Millisecond
<code>Z</code>	Time zone
<code>zzzz</code>	Full time zone name

Table 8: DateTimeFormatter Patterns

Common Exceptions

Exception	When thrown
NotSerializableException	Class doesn't implement Serializable
ClassNotFoundException	Class not found during deserialization
InterruptedException	Thread interrupted while waiting
IllegalArgumentException	Invalid argument passed
NullPointerException	Null object referenced
DateTimeException	Invalid date/time operation
IOException	I/O operation failed
ArithmeticException	Division by zero, overflow

Table 9: Common Exceptions

Concurrency Keywords

Keyword	Description
synchronized	Mutual exclusion lock
volatile	Guarantees visibility (not atomic)
transient	Excluded from serialization
final	Cannot be changed
static	Belongs to class, not instance

Table 10: Java Keywords

Module-Related Keywords

Keyword	Description
exports	Exports package to other modules
requires	Declares module dependency
provides	Declares service provider
uses	Declares service consumer
opens	Opens package for reflection

Table 11: Module Keywords

10 Practice Tips

Exam Tips

- **Read carefully!** Many questions have subtle differences
- **Check for compilation errors first** - many questions test this
- **Pay attention to modifiers:** `static`, `transient`, `volatile`
- **Watch for immutability** - Strings are immutable
- **Remember DST behavior** - dates can "repeat" or "skip"
- **Know serialization rules** - what gets serialized and what doesn't
- **Understand thread states** - which consume CPU and which don't
- **Be familiar with module system** - named vs automatic vs unnamed
- **Practice switch expressions** - arrow syntax and fall-through
- **Know Optional behavior** - when it's returned and how to handle it

Common Pitfalls

Important

Most Common Mistakes:

1. Assuming `String` methods modify the original
2. Forgetting that `transient` and `static` are not serialized
3. Not handling `InterruptedException`
4. Confusing `==` with `equals()`
5. Forgetting DST transitions in time zones
6. Not using `break` in traditional switch
7. Assuming `volatile` provides atomicity
8. Confusing `invokeAll` with `invokeAny`
9. Forgetting that `reduce()` without identity returns `Optional`
10. Not knowing automatic module naming rules

Tip

Quick Memory Aid:

- **S**erialization → **S**erializable, superclasses, **s**ubclass
- **D**ST → **D**ates repeat/skip
- **M**odules → **M**ain, **A**utomatic, **U**nnamed
- **L**ock → **l**ock (block), **t**ryLock (immediate)

11 Key Takeaways

Must Remember

1. **Enums** have private constructors
2. **Strings** are immutable
3. **reduce()** without identity returns **Optional**
4. **transient** fields are NOT serialized
5. **static** fields are NOT serialized
6. **InterruptedException** is **checked**
7. **volatile** = visibility, NOT atomicity
8. **tryLock()** returns immediately (non-blocking)
9. **Bottom-up migration** moves ALL modules to module path
10. Default methods need an instance (can't call on interface)
11. **truncatedTo()** rounds DOWN
12. DST causes time to repeat or skip